Проектна документација за темата: Дигитален идентитет со користење на Verifiable Credentials

1. *Вовед*

Овој проект претставува имплементација на систем за дигитален идентитет базиран на Verifiable Credentials (VC), дефиниран од World Wide Web Consortium (W3C). Целта е да се демонстрира издавањето на дигитален credentioal, криптографска заштита преку дигитален потпис, децентрализирана верификација и презентација на credential без повторна комуникација со issuer.

2. *Теоретска основа*

2.1 *Verifiable Credentials*

VC е дигитален документ кој содржи структурирани податоци, дигитално е потпишан од издавач, се верифицира криптографски и овозможува контрола на податоците од страна на сопственикот.

2.2 *Модел Issuer-Holder-Verifier*

Системот се базира на три улоги: issuer,holder и verifier.

2.2.1 *Issuer*

Издава и потпишува credentials.

```python
from flask import Flask, request, jsonify
import jwt, time
from pathlib import Path
app = Flask(__name__)

with open("issuer_private.pem", "rb") as f:
    PRIVATE_KEY = f.read()


@app.post("/issue")
def issue():
    data = request.get_json(force=True)
    now = int(time.time())

    vc_payload = {
        "type": "StudentIDCredential",
        "studentId": data["studentId"],
        "fullName": data["fullName"],
        "faculty": data["faculty"],
        "iat": now,
        "exp": now + 3600
    }

    token = jwt.encode(vc_payload, PRIVATE_KEY, algorithm="ES256")
    return jsonify({"vc_jwt": token})

@app.get("/")
def home():
    return "Issuer is running. Use POST /issue"

if __name__ == "__main__":
    app.run(port=5001, debug=True)
```

2.2.2 *Holder*

Го прима и чува credential.

```python
from flask import Flask, request, jsonify, render_template
import jwt, time, json, os
import requests

app = Flask(__name__)

HOLDER_PRIVATE = (open("holder_private.pem", "rb")).read()
HOLDER_PUBLIC = (open("holder_public.pem", "rb")).read()

WALLET_FILE = "wallet.json"
if not os.path.exists(WALLET_FILE):
    with open(WALLET_FILE, "w") as f:
        json.dump({"vcs": []}, f)

@app.get("/")
def index():
    with open(WALLET_FILE, "r") as f:
        wallet = json.load(f)
    return render_template("index.html", vcs=wallet["vcs"])

@app.post("/store")
def store():
    data = request.get_json(force=True)
    vc_jwt = data["vc_jwt"]

    with open(WALLET_FILE, "r") as f:
        wallet = json.load(f)
    wallet["vcs"].append(vc_jwt)
    with open(WALLET_FILE, "w") as f:
        json.dump(wallet, f, indent=2)

    return jsonify({"stored": True, "count": len(wallet["vcs"])})

```

```python
@app.post("/present")
def present():
    data = request.get_json(force=True)
    vc_jwt = data["vc_jwt"]

    # Get challenge (nonce) from verifier
    ch = requests.get("http://127.0.0.1:5004/challenge").json()
    nonce = ch["nonce"]

    now = int(time.time())
    vp_payload = {
        "type": "StudentIDPresentation",
        "vc_jwt": vc_jwt,
        "nonce": nonce,
        "iat": now,
        "exp": now + 300
    }

    vp_jwt = jwt.encode(vp_payload, HOLDER_PRIVATE, algorithm="ES256")
    return jsonify({"vp_jwt": vp_jwt, "holder_public_key_pem": HOLDER_PUBLIC.decode()})

if __name__ == "__main__":
    app.run(port=5002, debug=True)
```

2.2.3 *Verifier*

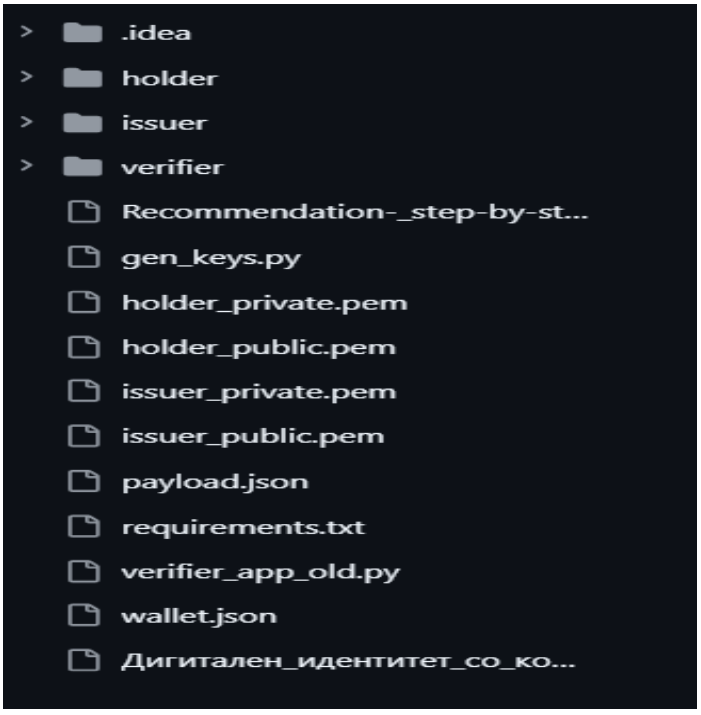
Го проверува credential без директна комуникација со issuer.

```python
from flask import Flask, request, jsonify, render_template
import jwt, time, secrets

app = Flask(__name__)

ISSUER_PUBLIC = open("issuer_public.pem", "rb").read()


NONCES = {}  # nonce -> exp timestamp

@app.get("/")
def index():
    return render_template("index.html")

@app.get("/challenge")
def challenge():
    ttl = 600
    nonce = secrets.token_urlsafe(16)
    exp = int(time.time()) + ttl
    NONCES[nonce] = exp
    return jsonify({"nonce": nonce, "expiresIn": ttl})

```

```python


@app.post("/verify")
def verify():
    data = request.get_json(force=True)
    vp_jwt = data["vp_jwt"]
    holder_public_pem = data["holder_public_key_pem"].encode()

    # 1) verify VP signature (holder)
    vp = jwt.decode(vp_jwt, holder_public_pem, algorithms=["ES256"])

    # 2) nonce check (anti-replay)
    nonce = vp.get("nonce")
    if not nonce or nonce not in NONCES:
        return jsonify({"valid": False, "error": "Missing/unknown nonce"}), 400
    if NONCES[nonce] < int(time.time()):
        return jsonify({"valid": False, "error": "Expired nonce"}), 400
    del NONCES[nonce]

    # 3) verify embedded VC signature (issuer)
    vc_jwt = vp["vc_jwt"]
    vc = jwt.decode(vc_jwt, ISSUER_PUBLIC, algorithms=["ES256"])

    if vc.get("type") != "StudentIDCredential":
        return jsonify({"valid": False, "error": "Wrong VC type"}), 400

    return jsonify({"valid": True, "vc_payload": vc})

if __name__ == "__main__":
    app.run(port=5004, debug=True)
```

3. ***Архитектура на системот***

Проектот е имплементиран со три независни Flask сервиси. Секој сервис работи независно и комуницира преку HTTP REST API повици.



4. *Користени технологии и алатки*
- Python
- Flask
- PyJWT
- Асиметрична криптографија (ES256)
-cURL

5. *Имплементирани сигурносни механизми*
- Автентикација
- Интегритет
- Replay заштита
- Временска валидност

6. *Имплементација*
6.1 *Генерирање на клучеви*
python gen_keys.py – ова генерира клучеви за issuer и holder

```python
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import serialization

def write_pair(priv_name: str, pub_name: str):
    private_key = ec.generate_private_key(ec.SECP256R1())
    public_key = private_key.public_key()

    private_pem = private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    )
    public_pem = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )

    with open(priv_name, "wb") as f:
        f.write(private_pem)
    with open(pub_name, "wb") as f:
        f.write(public_pem)

write_pair("issuer_private.pem", "issuer_public.pem")
write_pair("holder_private.pem", "holder_public.pem")
print("Generated issuer_*.pem and holder_*.pem")
```

6.2 *Старт на системот*
- за holder: python holder/app.py и работи на http://127.0.0.1:5002/
- за verifier: python verifier/app.py и работи на http://127.0.0.1:5004/
- за issuer: python issuer/app.py и работи на http://127.0.0.1:5001/

7. *Процес*
7.1 Издавање на VC
curl -X POST http://127.0.0.1:5001/issue \
-H "Content-Type: application/json" \
-d '{"studentId":"201234","fullName":"Ime Prezime","faculty":"FINKI"}'

Со ова се креира JWT payload, се потпишува со приватен клуч и враќа vc_jwt.

7.2 *Складирање кај Holder*

```
curl -X POST http://127.0.0.1:5002/store \
-H "Content-Type: application/json" \
-d '{"vc_jwt":"PASTE_VC_JWT_HERE"}'
```

Се зачувува VC во wallet.json

7.3 *Креирање на Verifiable Presentation(VP)*

```
curl -X POST http://127.0.0.1:5002/present \
-H "Content-Type: application/json" \
-d '{"vc_jwt":"PASTE_VC_JWT_HERE"}'
```

Тука во овој код holder креира VP со VC, потпишува VP со својот приватен клуч и враќа vp_jwt.

7.4 *Верификација*

```
curl -X POST http://127.0.0.1:5004/verify \
-H "Content-Type: application/json" \
-d
'{"vp_jwt":"PASTE_VP_JWT_HERE","holder_public_key_pem":"PASTE_HOLDER_PUBLIC_KEY_PEM_HERE"}'
```

Во овој код Verifier го проверува потписот на holder, проверува потпис на issuer и проверува истекување на важност.

Изработиле:
Мартина Апостолоска 193184
Јована Лазарова 221221
Линк од проектот: [GitHub - apostoloska21/digital_identity_using_verifiable_creditentials · GitHub]